物聯網應用與資料分析 HW2 — AI 輔助對話紀錄

工具：Claude（Claude Code 桌面版）· 只保留雙方對話文字，略去程式執行過程與工具輸出 · 共 60 段

我

https://lms2020.nchu.edu.tw/course/homework/75217

這是我的物聯網應用與資料分析 作業對話框

Claude（AI）

I opened the link in the built-in browser, but it sent me to the course's info page instead of the assignment. The browser isn't signed in to iLearning, and I'm not allowed to type your password for you.

To keep going:

1. **Open the browser pane:** press **Ctrl+Shift+B**, or close whatever is open in the side panel and click the globe icon.
2. **Sign in to iLearning 3.0** with your school account. You only need to do this once; the browser stays signed in.
3. **Tell me what you want me to do with this assignment.** For example:
4. Read the assignment instructions and summarize them.
5. Help you write the code, report or data analysis it asks for.
6. Check the due date and what format to submit in.

Once you're signed in, I'll open the assignment page again and read it. I won't submit or upload anything until you confirm.

我

他要cloudflare驗證

Claude（AI）

I'm not allowed to complete the Cloudflare check for you, so you'll need to do that step yourself:

1. In the browser pane that's open now (the school login page, ccidp.nchu.edu.tw), check the Cloudflare verification box.
2. Sign in with your school account.
3. Reply "done" (or 好了) when you're in, and tell me what you want done with the assignment.

Once you're signed in, I'll open the assignment page again and read it.

我

[iLearning 3.0](https://lms2020.nchu.edu.tw/)

* [系統討論區](https://lms2020.nchu.edu.tw/systemDiscuss)
* [我的首頁](https://lms2020.nchu.edu.tw/dashboard)
* [（姓名已隱藏）](https://lms2020.nchu.edu.tw/course/homework/75217#)
* [繁體](https://lms2020.nchu.edu.tw/course/homework/75217#)

[物聯網應用與資料分析](https://lms2020.nchu.edu.tw/course/46977) (7716)
物聯網應用與資料分析
老師: 陳煥 HUAN CHEN
助教: 詹凱宇 CHAN,KAI-YU , 李宗諺 LEE,TSUNG-YEN
[私人留言](https://lms2020.nchu.edu.tw/course/privateComment/46977)

* [課程活動](https://lms2020.nchu.edu.tw/course/46977)
* [公告](https://lms2020.nchu.edu.tw/course/bulletin/46977)
* [行事曆](https://lms2020.nchu.edu.tw/course/calendar/46977)
* [課程資訊](https://lms2020.nchu.edu.tw/course/info/46977)
* [上課教材](https://lms2020.nchu.edu.tw/course/material/46977)
* [討論區](https://lms2020.nchu.edu.tw/course/discuss/46977)
* [線上同步教室](https://lms2020.nchu.edu.tw/course/onlineMeeting/46977)
*
* [作業 (6)](https://lms2020.nchu.edu.tw/course/homeworkList/46977)
* [測驗](https://lms2020.nchu.edu.tw/course/examList/46977)
* [問卷](https://lms2020.nchu.edu.tw/course/questionnaireList/46977)
* [互評](https://lms2020.nchu.edu.tw/course/reviewList/46977)
* [即時回饋](https://lms2020.nchu.edu.tw/course/feedbackList/46977)
*
* [小組專區](https://lms2020.nchu.edu.tw/course/teamStageList/46977)
* [出缺勤](https://lms2020.nchu.edu.tw/course/attendanceRecord/46977/?（已隱藏）)

1. [物聯網應用與資料分析](https://lms2020.nchu.edu.tw/course/46977)
2. [作業](https://lms2020.nchu.edu.tw/course/homeworkList/46977)
3. DIC2-多元線性回歸 50-startup and boston housing

DIC2-多元線性回歸 50-startup and boston housing
類型
個人作業
開放繳交
2026-10-12 00:00
繳交期限
2026-10-14
已繳交
0人
允許遲交

否
成績比重
5%
評分方式
直接打分數
說明
do two problems but choose one of the topic to upload
第二個直接套skill or workflow
Result
https://huanchen1107.github.io/AIoT-DIC2-MultipleLR/
加總分 1分:
1. 有下拉視窗呈現兩種problem
加總分2分
1. 可任意輸入 Dataset (such as https://raw.githubusercontent.com/Avik-Jain/100-Days-Of-ML-Code/master/datasets/50_Startups.csv)
🌐 您的 GitHub Pages 預計公開網址：
👉 https://huanchen1107.github.io/AIoT-DIC2-MultipleLR/
⚙️ 請在 GitHub 進行最後 1 步啟用：
已為您建立並推播了 GitHub Pages 的官方自動部署工作流（`.github/workflows/pages.yml`），請按照以下步驟開啟：

1. 點擊開啟倉庫的 Pages 設定頁： 👉 https://github.com/huanchen1107/AIoT-DIC2-MultipleLR/settings/pages
2. 在 Build and deployment 區塊：
  * 方式一（推薦）：在 Source 下拉選單選擇 `GitHub Actions`。
    * 系統會自動觸發剛才推送的 Actions 工作流，約 30 秒內自動完成部署上線！
  * 方式二（傳統）：在 Source 選擇 `Deploy from a branch`：
    * Branch 選擇 `main`，資料夾選擇 `/ (root)`，點擊 Save。
3. 稍等約 1 分鐘後重新整理，即可看到綠色勾勾與發布成功的公開網址！

附件

* [image.png (654.6 KB)](https://lms2020.nchu.edu.tw/filedownload/2864273)

附件

* 1.
[image.png (63.2 KB)](https://lms2020.nchu.edu.tw/filedownload/2865310)
* 2.
[image.png (91.8 KB)](https://lms2020.nchu.edu.tw/filedownload/2865311)

© Copyright since 2020 by National Chung Hsing University.
All rights reserved.

● 本站由 [教學資源暨發展中心](http://oaa.nchu.edu.tw/zh-tw/cdtl) 管理●
線上： 1368 人

Claude (AI)

Now testing it in the browser pane with both datasets.

I built the assignment page at index.html and ran both datasets in the browser pane. The results match the standard answers for these datasets:

Problem	R^2	Main results
50 Startups	0.9508	R&D Spend coefficient 0.806
Boston Housing	0.7406	Intercept 36.46

These were fitted on all the rows (test ratio set to 0). The page itself defaults to an 80/20 train/test split.

How it covers the two scoring points: - **1 point:** a dropdown switches between 50 Startups and Boston Housing, plus a third "自訂 Dataset" option. - **2 points:** you can paste any CSV URL. It reads the columns automatically, lets you pick the target (y) and the features (X), and turns text columns like State into 0/1 columns.

The page also shows Train/Test R^2, RMSE, a table of coefficients and a chart of actual vs. predicted values.

Still to do, due today: it needs to go up on GitHub Pages. The GitHub command-line tool isn't installed here, so I can't create the repository for you. Either: - **You do it:** create an empty public repository on GitHub, send me its URL, and I'll push to it with git. Then in that repository's Settings → Pages, choose `Deploy from a branch`, branch `main`, folder `/ (root)`. - **Or upload by hand:** on the new repository's page, click "uploading an existing file" and drag in `index.html`.

When it's live, the address will be `https://<your account>.github.io/<repository name>/`. Submit that on iLearning.

The CSV reader doesn't handle commas inside quoted values. Neither preset dataset has any, so it only matters for a custom dataset.

我

但好醜喔老師給的圖示這樣

Claude (AI)

Got it. I'll redesign it to match the teacher's example: a purple header banner, 5 feature-selection methods, k=1..p buttons, metric cards, RMSE/R^2 charts, coefficient table with p-values and VIF, a live predictor and a residuals table. The 50 Startups preset will use the same 80/20 split as sklearn's `random_state=0`, so the numbers should match the teacher's (k=2 → RMSE 8198.80).

Every number matches the teacher's screenshot exactly: RMSE $8,198.80, R^2 0.9474, Adj R^2 0.9324, MAE $6,886.59, F 349.0, and the same coefficients. Now checking Boston and the look.

Testing the Boston Housing dataset and the "Compare All" mode next.

Slight horizontal overflow at narrow width; fixing.

No more page overflow (the wide tables scroll inside their own boxes). One last look at the lower half:

I redesigned index.html to look like the teacher's example. I ran it in the browser pane: on 50 Startups the numbers match the teacher's screenshot exactly, and Boston Housing works too.

	Teacher's example	Yours
Test RMSE (k=2)	$8,198.80	$8,198.80
R^2 / Adjusted R^2	0.9474 / 0.9324	0.9474 / 0.9324
MAE / F-statistic	$6,886.59 / 349.0	$6,886.59 / 349.0
Model	46,346.34 + 0.7886·R&D Spend + 0.0326·Marketing Spend	same

The numbers match because the 50 Startups preset uses the same test rows as the usual sklearn split (`random_state=0`). Boston Housing and custom CSVs use their own random 80/20 split.

Layout (same order as the teacher's): - Purple header banner. - 5 selection methods (Lasso, Forward SFS, RFE, ElasticNet, Exhaustive) plus "Compare All 5 Methods". - k=1…p buttons, with the best k in green (★) and the full model in yellow (⚠). - 5 metric cards. - RMSE and R^2 line charts. - The regression equation. - Coefficient table with p-values, 95% confidence intervals and VIF. - A live predictor with sliders. - Residuals table.

Scoring points: - **1 point:** the dropdown switches between 50 Startups and Boston Housing. - **2 points:** you can paste any CSV URL, choose the target column, and text columns are turned into 0/1 columns automatically.

One difference from the teacher's example: for k=3 to 5 the teacher's model adds the state columns. My methods pick Administration as the third feature, because it fits the training data better. k=1, k=2 (the best one) and k=5 still match.

Still to do, due today: put it on GitHub Pages. Create an empty public repository on GitHub, upload `index.html`, then go to Settings → Pages and choose `main` / `/ (root)`. Or send me the repository URL and I'll push it with git.

我

幫我推到GitHub

Claude (AI)

Git on this machine already has saved GitHub credentials, so pushing will work. I still need to create the repository; checking whether the browser is signed in to GitHub:

The code is committed locally. Pushing just needs a repository on GitHub, and the built-in browser isn't signed in to GitHub. I'm not allowed to enter your password, so please do this:

1. Sign in to GitHub in the browser pane on the right. It's already open at the sign-in page.
2. Go to https://github.com/new and create a repository:
3. Name: `AIoT-DIC2-MultipleLR` (or any name you like)
4. Visibility: **Public**. GitHub Pages on a free account needs a public repository.
5. **Leave** "Add a README" **unchecked**, so the repository starts empty.
6. Send me the repository URL, for example `https://github.com/yourname/AIoT-DIC2-MultipleLR`.

Once I have the URL, I'll push with your saved git credentials and walk you through switching on Pages.

我

好了，建好了https://github.com/Ollyye16/IoT-DIC2-MultipleLR

Claude (AI)

Pushed: `index.html` is now on the `main` branch of https://github.com/Ollyye16/IoT-DIC2-MultipleLR.

GitHub Pages isn't switched on yet. I've opened your repository's Pages settings in the browser pane. Switching it on publishes the site, so I need your OK before changing that setting:

- Under Branch, choose **`main`** and folder **`/ (root)`**, then click **Save**.

Reply "好" and I'll do it, or do it yourself in the pane (it takes a few seconds). After about a minute the site will be at:

👉 **https://ollyye16.github.io/IoT-DIC2-MultipleLR/**

Submit that URL on iLearning. Once it's live, I can open it to check it loads.

我

50-startup and boston housing 為甚麼老師是用python寫backend邏輯呢?

Claude (AI)

The teacher most likely didn't use Python as the website's backend. Python was for the analysis; the web page itself runs in the browser.

How to tell: 1. **GitHub Pages can't run Python.** It only serves static files (HTML/CSS/JS), so a site hosted at `github.io` has no backend at all. 2. **The footer of the teacher's page** says "Built with pure Vanilla HTML, CSS, JavaScript, and Chart.js." 3. **The other two screenshots are Python output.** The RMSE/R^2 table is a pandas DataFrame, and the two line charts are matplotlib's default style. Those are the analysis results from a notebook (Colab or Jupyter), not the web page.

So the teacher's workflow was probably: 1. **Analysis in Python** with `pandas` + `scikit-learn` (train/test split, OLS, Lasso, RFE, SFS) to get the "best is 2 features, RMSE 8198" result. That's the data-analysis part of the CRISP-DM process, and Python is the standard tool for it. 2. **Then a web page** using the "skill or workflow" from the assignment description, probably an AI agent that turned the analysis into HTML/JS and deployed it to Pages.

Why Python for the analysis: - sklearn already has `train_test_split`, `Lasso`, `RFE` and `SequentialFeatureSelector`, so each is one line. - `statsmodels` gives the p-values, confidence intervals and VIF directly. - It's the course's main tool (資料分析), and the numbers are easy to check.

What this means for you: your web page reimplements the same math in JavaScript, and the numbers match sklearn exactly, so the web part is fine. But the assignment says "do two problems". If the teacher expects to see the analysis process, it's worth also handing in a Python notebook with both 50 Startups and Boston Housing. I can write that `.ipynb` and push it to the same repository. Do you want me to?

我

我知道不行但HTML很怪

Claude (AI)

Agreed, writing linear regression by hand in JavaScript is unusual. In practice nobody does that; everyone uses sklearn. You can still use real Python, in two ways:

Option A: Pyodide (Python inside the web page). Still on GitHub Pages. - The page loads Pyodide (Python compiled to run in the browser) and runs **real** `pandas` + `scikit-learn` code there. - The analysis logic becomes a `.py`-style block with `Lasso`, `RFE`, `SequentialFeatureSelector` and `statsmodels`, the same as in your notebook. - The current design stays, and the URL stays `ollyye16.github.io/...`. - Downside: the first load takes about 10–20 seconds to download Python and sklearn.

Option B: Streamlit (the whole app in Python). - One `app.py` builds the whole interface, with no HTML at all, and it's very common in data-analysis courses. - It's deployed to Streamlit Community Cloud, not GitHub Pages, so the URL becomes `xxx.streamlit.app`. Deploying needs you to sign in to Streamlit with your GitHub account. - Downside: the assignment shows a github.io address. Whether a different address is accepted is the teacher's call.

I recommend A: it keeps the github.io address the assignment shows and puts the core in Python, so I can switch the existing page over quickly.

Want me to switch to A? Or did "HTML很怪" mean the page itself looks wrong somewhere? If so, tell me which part.

我

https://github.com/huanchen1107/AIoT-DIC2-MultipleLR

阿老師為什麼有用python?

Claude (AI)

I looked at the teacher's repository. The teacher built two versions of the same page:

1. The real Python backend: Flask, deployed on Vercel - `app.py` is a Flask app with API endpoints: `GET /api/models` returns all the model results, and `POST /api/predict` calculates a live prediction. - sklearn (`LassoCV`, `RFE`, `SequentialFeatureSelector`) and `statsmodels` (p-values, VIF) do the calculations. - `api/index.py` and `vercel.json` deploy it to **Vercel**, which can run Python. That's the address in the repository's About section: `a-io-t-dic-2-multiple-lr.vercel.app`.

2. GitHub Pages: static, with results computed in advance - The Python scripts `main.py` and `feature_progression_analysis.py` compute everything first and save it to `model_data.json` and `static/model_data.js`. - The github.io page **just reads that saved data and draws it with Chart.js**. No Python runs there.

So "GitHub Pages can't run Python" is still true. The teacher ran Python ahead of time and uploaded the results, and set up a Vercel version for live Python.

The catch: the saved-results approach only works for the fixed 50 Startups data. For the 2-point requirement (paste any dataset URL), something has to calculate in real time, which means a backend like Flask on Vercel, or JavaScript or Pyodide in the browser. Your current version computes in the browser with JavaScript, so it handles any URL on GitHub Pages without a backend.

If you want to match the teacher's setup, I can add to your repository: - `app.py` (Flask + sklearn + statsmodels), taking a dataset URL and returning the model results. - `api/index.py` + `vercel.json` + `requirements.txt`. - Your existing `index.html` would call the Flask API when it's available and fall back to the in-browser calculation when it isn't. GitHub Pages keeps working either way.

You'd then need to import the repository on vercel.com yourself (sign in with GitHub, click Import). Want me to build that?

我

你可以說中文嗎

Claude (AI)

好的，之後都用中文回覆。

老師為什麼用 Python？

我看了老師的 repo，他其實做了兩個版本：

1. 真正的 Python 後端：Flask，放在 Vercel - `app.py` 是一個 Flask 網站，有 API： - `GET /api/models`：回傳所有模型結果 - `POST /api/predict`：即時算出預測值 - 計算用 sklearn（Lasso、RFE、SFS）和 statsmodels（p-

value、VIF）。 - 部署在 **Vercel**，它可以跑 Python。網址是 `a-io-t-dic-2-multiple-lr.vercel.app`。

2. GitHub Pages 版：靜態網頁，結果先算好 - 先在自己電腦跑 Python（`main.py`、`feature_progression_analysis.py`），把結果存成 `model_data.json` / `model_data.js`。 - github.io 上的網頁**只是讀這份算好的資料，再用 Chart.js 畫出來**，網頁上沒有跑 Python。

所以「GitHub Pages 不能跑 Python」還是對的。老師是先用 Python 算好再上傳，另外架了一個 Vercel 版來即時跑 Python。

有個問題： 預先算好的做法只適用固定的 50 Startups 資料。加分項目要「可任意輸入 Dataset URL」，就一定要即時計算，有三種方式： - 後端，例如 Vercel 上的 Flask - 瀏覽器裡跑 JavaScript ← 你現在的版本，所以任何 URL 都能用 - 瀏覽器裡跑 Python（Pyodide）

想跟老師的架構一樣的話，我可以幫你在 repo 加上： - `app.py`（Flask + sklearn + statsmodels），輸入資料集 URL，回傳模型結果 - `api/index.py`、`vercel.json`、`requirements.txt` - `index.html` 改成：Vercel 後端在線就呼叫 Flask API，不在線就自動改用瀏覽器內計算，所以 GitHub Pages 版照樣能用。

最後要你自己到 vercel.com 用 GitHub 登入，按 Import 匯入這個 repo。

要我做嗎？另外，你的 GitHub Pages 還沒開啟。到 repo 的 Settings → Pages 選 `main` / `/ (root)` → Save；或回我「好」，我幫你在瀏覽器裡設定。

> **我**
>
> 好，幫我做 Flask 版本 但我做出來要跟老師網頁都長一模一樣嗎?

Claude（AI）

不用一模一樣。評分只看兩項：

- **1 分**：有下拉選單切換兩種 problem
- **2 分**：可以任意輸入 Dataset URL

老師的網頁只是參考範例。你現在的版本兩項都有，而且做得比範例多（範例只能跑固定的 50 Startups）。完全照抄反而可能被當成抄襲，保留自己的樣子比較好。

接下來做 Flask 版本，先確認本機的 Python 環境：

Python 後端算出的結果跟 JavaScript 版完全一樣（RMSE 8198.80、F 349.04、VIF 2.10）。接下來改前端：能連到 Flask 就用 Python 算，連不到就用瀏覽器內的 JS 算。

Flask API 在本機測試成功。現在把套件版本固定（避免 Vercel 裝到不同版本），再確認前端能正確顯示 Python 回傳的結果。

The JS path works, but the message says "3 candidate features" instead of 5, so something's wrong. Checking:

Found it: an edit I made earlier put a comment in front of `.forEach(...)` on line 370, so the State dummy columns were never created. Fixing:

已推上 GitHub：https://github.com/Ollyye16/IoT-DIC2-MultipleLR

新增的檔案： - `app.py`：Flask 後端。`POST /api/analyze` 用 scikit-learn（Lasso/ElasticNet 路徑、RFE、LinearRegression）和 statsmodels（OLS、p-value、VIF）計算全部模型。本機跑過，結果和老師的一樣（RMSE 8198.80、F 349.04）。 - `api/index.py`、`vercel.json`、`requirements.txt`：Vercel 部署用。 - `index.html`：連得到後端就用 Python 算，畫面顯示「 Python (Flask + scikit-learn + statsmodels)」；連不到（例如 GitHub Pages）就改用瀏覽器內 JavaScript 算，結果相同。 - `README.md`：說明架構、執行方式和結論。

修掉一個 bug： 上一版推上去的網頁，50 Startups 的 State 欄位不見了（只剩 3 個特徵），這版已修正。

還沒驗證： Flask API 回傳的資料我用 curl 確認過，也在瀏覽器模擬過 Python 路徑的顯示（用 JS 的結果冒充），兩邊都正常。但「網頁實際呼叫 Flask」這段還沒真正跑過，因為內建瀏覽器連不到我本機開的伺服器。你可以自己跑 `python app.py`，開 http://localhost:5000/ 看看。

⚠️ **要跟你說一件事：** 關掉本機 Flask 時，我先下了一個太寬的指令 `taskkill /IM python.exe`，它可能把你電腦上**所有**正在跑的 Python 程式都關掉了。如果你剛才有開 Jupyter 或其他 Python 程式，請確認一下，抱歉。

接下來要你做的： 1. **開 GitHub Pages**（作業要交的網址）：到 repo 的 Settings → Pages，Branch 選 `main`、`/(root)` → Save。或回我「好」，我在瀏覽器幫你設定。 2. **（選擇性）部署 Vercel**，讓線上版也跑 Python：到 vercel.com 用 GitHub 登入，按 Add New → Project，選這個 repo，按 Deploy。拿到網址後告訴我，我把它填進 `index.html` 的 `VERCEL_API`，這樣 GitHub Pages 版也會改用 Python 計算。

我

為什麼RMSE by Number of Features 13個 我點了不同的K圖片也沒變

Claude（AI）

那張圖本來就是畫出 k = 1…13 **每一個** k 的 RMSE（一條折線），所以點不同的 k，整條線不會變；會變的是上面的數字卡、方程式、係數表和預測器。不過圖上目前沒標出你選了哪個 k，看起來就像沒反應。我讓選到的 k 在圖上標出來：

畫面還是舊版，我重新載入最新的檔案：

預覽視窗沒更新，我直接推上 GitHub，再用線上預覽檢查：

點不同的 k，圖上的點會跟著動了，已推上 GitHub。

原因： 這兩張圖本來就是把 k = 1…13 **每一個** k 的 RMSE（和 R^2）都算出來，畫成一條線，用來看「特徵數量越多，誤差怎麼變」。所以不管你點哪個 k，線本身都一樣。點 k 會改變的是上面的數字卡、方程式、係數表和預測器。但之前圖上沒標出你選了哪個 k，看起來就像沒反應。

修改後： - 🟢 綠點：最佳的 k（RMSE 最小） - 🟠 大的橘色圈：你目前點選的 k，點別的 k 會跟著移動 - 圖的副標題加上了這兩個符號的說明

我在 Boston Housing 點 k = 5 測過，橘色圈正確出現在 k = 5 的位置。

順便說明這張圖： Boston 的 RMSE 從 k = 1 到 7 下降很快，之後幾乎持平，k = 11 最低（4.67）。代表大約 7 個特徵以後，再加特徵幫助就很小了，報告裡可以這樣寫。

GitHub Pages 還沒開，記得到 Settings → Pages 選 `main` / `/ (root)` → Save，或回我「好」我幫你設。

我

老師說跟他做太像了 應該是前端渲染太像

Claude（AI）

Repo 沒有未提交的變更。我把前端整個重新設計，換掉老師範例的視覺和版面，計算的部分（JS 和 Flask）不動。先看一下目前的程式碼：

計算部分（第 192–341 行）我會抽到獨立的 `model.js`，`index.html` 只放全新設計的介面：

預覽視窗是用 data: 網址開的，讀不到同資料夾的 `model.js`。Flask 也要多開一個路由提供 `model.js`。先修 Flask，再推上去用線上預覽測試：

數字都正確。來看畫面：

側欄在窄畫面會疊在上面，正常。用桌機寬度看主畫面：

我

（選項回答）確認一下「換皮」的意思：要保留哪個版本的結構？ → 回到上一版結構，只換外觀

Claude（AI）

了解，回到上一版的結構，只換外觀（配色、字型、文字）。先還原上一版：

Restored. Now the reskin — reading the style and page markup:

Now the script side: method names, colors and the English text inside render:

Python script for all the text and color replacements:

The heredoc got mangled by the shell, so I'll put it in a file instead:

30 處都換好了，也沒有殘留的老師範例用語。推上去，用線上預覽檢查：

外觀都正常。只剩表格捲軸還是白色，在 CSS 加上 `color-scheme: dark` 讓捲軸和表單元件也變深色：

改好了，已經推上 GitHub。版面和操作都回到上一版，只換了外觀。

沒動的部分： 方法按鈕、k = 1…5 按鈕、統計卡、兩張折線圖、方程式、係數表、即時預測、殘差表、「五種方法一起比較」，還有所有計算結果（RMSE $8,198.80 等）。

換掉的部分：

	老師範例	你的版本
主題	白底 + 紫色漸層	深藍底 + 青綠色，重點用金色
標題	Multiple Linear Regression Explorer	迴歸實驗室 · 50 Startups
頂部橫幅	紫色大橫幅 + 英文說明	深色卡片 + 金色左邊線，中文說明（自動帶入結果）
文字	英文（Sweet Spot、Occam's Razor…）	全部中文（最佳、全部特徵、顯著/不顯著、離群/正常…）
字型	Inter / JetBrains Mono	Noto Sans TC / IBM Plex Mono
圖上的點	綠點 = 最佳	金點 = 最佳，粉紅圈 = 目前選的 k
步驟標籤	0. / 1. / 2. 英文大寫	①②③ 圓圈編號

另外，上一次那版（側欄 + 分頁 + 熱力圖）已經撤掉了，`model.js` 也刪了，Flask 後端維持不變。

還沒做： GitHub Pages 還沒開啟。到 repo 的 Settings → Pages，Branch 選 `main`、`/ (root)` → Save，網址就是 https://ollyye16.github.io/IoT-DIC2-MultipleLR/ 。或回我「好」，我幫你在瀏覽器設定。

我

HW2: Multiple Linear regression
類型
個人作業
開放繳交
2025-10-15 00:00
繳交期限
2026-11-04
已繳交
22人
允許遲交
是
成績比重
10%
評分方式
直接打分數
說明

========加分 ==========
加總分 1分:
1. 有下拉視窗呈現兩種problem
加總分2分
1. 可任意輸入 Dataset (such as https://raw.githubusercontent.com/Avik-Jain/100-Days-Of-ML-Code/master/datasets/50_Startups.csv)
======== ======== ========

一、作業目標：

本次作業延伸自 HW1，目標是讓同學能夠以實際資料集進行「多元線性回歸 (Multiple Linear Regression)」的完整分析，並遵循 CRISP-DM 流程完成從資料理解、建模到評估的全過程。

二、作業內容：

1. 資料來源

至 Kaggle 選擇一個具有>13 個特徵 (features) 的公開資料集。

類型不限（可為房價預測、醫療、車輛效能等主題）。

請明確標示資料集來源與連結。

2. 分析任務

使用線性回歸 (Linear Regression) 模型進行預測。

可嘗試單純線性回歸、多元線性回歸或 Auto Regression。

必須執行 特徵選擇 (Feature Selection) 與 模型評估 (Model Evaluation)。

結果部分需包含請提供預測圖(加上信賴區間或預測區間)

3. CRISP-DM 流程說明

Business Understanding

Data Understanding

Data Preparation

Modeling

Evaluation

Deployment

4. AI協助要求

所有與 ChatGPT 的對話請以 pdfCrowd 或其他方式須匯出為 PDF

請使用 NotebookLM 對網路上同主題的解法進行研究，並撰寫一份 100 字以上的摘要，放入報告中。

請在報告中明確標示「GPT 輔助內容」與「NotebookLM 摘要」

5. 繳交內容

主要繳交: 跟 DIC2 一樣github and web

optional :對話內容, 資訊圖表

==========

三、評分標準

主要繳交: 跟 DIC2 一樣github and web

optional :對話內容, 資訊圖表 , 報告書

不夠事後可再補 1次

==== 補充 ====

list kaggle multiple linear regression problems , choose features > 13, < 40

Kaggle Multiple Linear Regression Problems (14–39 Features)

你希望找適合 Multiple Linear Regression（多元線性回歸） 的 Kaggle 題目，條件為：

13<Number of Features<4013<Number of Features<40

也就是 14–39 個輸入特徵，不包含 Target（預測目標）。

建議優先選擇連續數值預測（Regression）、可使用 sklearn.LinearRegression、適合進行 Feature Selection、MSE、RMSE、R^2 與五大回歸假設檢驗的資料集。

特別注意：Kaggle 上常見的 House Prices — Advanced Regression Techniques 有 79 個原始特徵，不符合你的原始特徵數限制，但可以透過特徵選擇建立 14–39 特徵的練習版本。

www.kaggle.com&sz=32
Kaggle
+1
Recommended Kaggle Regression Problems
以下區分「原始特徵符合條件」與「需要 Feature Selection 才符合條件」。

#

Kaggle Problem

Features

Target

Suitability

1

Predict the Flood

20

FloodProbability

Excellent

2

Flood Prediction Dataset

20

FloodProbability

Excellent

3

Istanbul Apartment Prices 2026

29 or fewer*

Price

Good

4

Swiss Housing Price Prediction

30

House price

Good; restricted access

5

Human–AI Interaction

20

Trust score

Excellent; synthetic

6

House Prices — Advanced Regression

79 → select 20–30

SalePrice

Excellent for feature selection

*Istanbul 資料集描述為 30 欄，包含價格等欄位；精確輸入特徵數需檢查 CSV。上述資料集的特徵數以來源描述為準，類別編碼後可能增加。
www.kaggle.com&sz=32
Kaggle
+5

My Top 3 for Teaching
China’ s Yellow River will be choked by ice jams due to global warming, study finds | South China Morning Post
1. Predict the Flood

20 numerical features, continuous target. Best for teaching OLS, MSE, R^2, residual analysis and regression assumptions.

Aerial View of Houses
2. Istanbul Apartment Prices

Suitable for categorical encoding, feature selection, and housing-price regression. Check the dataset columns before modeling.

AISCA
3. Human–AI Interaction

20 original features, 10,100 samples. Useful for studying coefficients, categorical features and multicollinearity.

Suggested Assignment: CRISP-DM + Feature Selection
For any selected dataset, students can complete:

Business Understanding: Define the continuous prediction target.

Data Understanding: Verify sample size, feature count, missing values and correlations.

Data Preparation: Train/test split, missing-value handling and categorical encoding.

Modeling: Train LinearRegression, Ridge and Lasso.

Evaluation: Compare MAE, MSE, RMSE, R^2 and Adjusted R^2; check the five regression assumptions.

Deployment: Predict new samples and explain the model.

For Feature Selection, compare SelectKBest, RFE, Sequential Feature Selection, Lasso and permutation importance, selecting 14, 20, 25 or 30 features using training data only.

My first choice is Predict the Flood, because its 20 numerical features meet your constraint directly and avoid the extra complexity of categorical encoding.

（選項回答）要用哪個 Kaggle 資料集當主題？（下拉選單的第二個 problem 可以保留 50 Startups 或 Boston）→ Life Expectancy (WHO) (Recommended)
（選項回答）網站和 repo 怎麼處理？ → 複製 DIC2 開新 repo (Recommended)

Claude（AI）

先找 Life Expectancy 資料集可以直接讀的公開網址：

那個網址只是 Git LFS 的指標檔，不是真正的資料。改從 LFS 的實際檔案位置下載：

後端改好了，用壽命資料測試：

接著加「預測區間圖」卡片，並修正即時預測器遇到缺值時的範圍計算：

先在預覽視窗測試。預覽讀不到相對路徑的 `data/`，所以測試時用同一份資料的線上網址代替：

JS 和 Python 的結果完全一致（Forward 每個 k 的 RMSE 都相同）。但瀏覽器內要算 38 秒，太慢了，找一下瓶頸：

預覽還是舊版。關掉分頁重新載入：

預覽面板目前是隱藏的，沒辦法自動重新載入。改成直接啟動 Flask 開發伺服器，順便測試「網頁呼叫 Python 後端」這條路（DIC2 時沒能真正測過）：

這台電腦載入 Python 套件非常慢（debug 模式還會再啟動一次），伺服器還在啟動中，等它完成。

伺服器還在啟動。先寫 `analysis.py`，給報告產生圖表和數字（跑一次 CRISP-DM 流程，輸出 PNG 圖和結果表）：

等待期間把網頁上的「DIC2」字樣改成 HW2：

Flask 還沒啟動完成，等它的通知。

離線分析跑完了，結果跟網頁一致（RFE k=16，RMSE 4.064，PI 涵蓋率 93.7%）。瀏覽器可以連到本機 Flask，來測網頁實際呼叫 Python 後端：

瀏覽器內計算從 38 秒降到 0.86 秒。再截一次圖：

圖表正常：黃點是實際值，粉紅點是落在預測區間外的樣本。圖例名稱「上界」容易誤會，改成直接寫區間名稱，同時檢查有沒有超出畫面寬度：

沒有超出畫面（之前那張是截圖縮放造成的）。看一下 `analysis.py` 產生的報告用圖：

圖表重新產生完成。關掉本機 Flask（這次只關這個伺服器的程序，不碰其他 Python 程式）：

HW2 的網站和 CRISP-DM 報告都做好了，在 `~/AIoT-HW2-MultipleLR`。照你的規則我沒有 commit / push，檔案都還沒提交。網頁走 Python 後端和瀏覽器內 JS 兩條路都在本機跑過，結果一致；`analysis.py` 也跑過，報告裡的數字都是從它的輸出核對過的。

主要結果（Life Expectancy，測試集 586 筆）： - 最佳是 RFE 選 16 個特徵：RMSE 4.06 歲，R^2 0.83。 - 只用 4 個特徵（Schooling、Adult Mortality、HIV/AIDS、Diphtheria）就有 R^2 0.79，報告裡當作精簡版本討論。 - 95% 預測區間大約 ±8 歲，有 93.7% 的實際值落在區間內，接近理論的 95%。 - 有兩個值得寫的發現： - infant deaths 和 under-five deaths 高度共線（VIF 約 200），讓 RFE 在 k=1、2 時選錯特徵。 - 模型會高估低壽命國家。

作業要求對照：

要求	狀態
Kaggle 資料集、14–39 個特徵、標示來源	✅ Life Expectancy (WHO)，20 個特徵
多元線性迴歸 + 特徵選擇 + 模型評估	✅ 5 種方法，k=1…20
預測圖加信賴區間 / 預測區間	✅ 網頁上有互動圖，`figures/` 裡有 PNG
CRISP-DM 六個步驟	✅ 寫在 `README.md`
加分：下拉選單兩種 problem、任意 Dataset	✅ Life / 50 Startups / Boston / 自訂網址
GPT 對話 PDF、NotebookLM 摘要	⚠️ 要你自己做，README 最後留了兩個 TODO 欄位

跟 DIC2 比，新增和修改的地方： - **新功能**： - 缺值處理：刪掉目標值缺的 10 筆，其餘缺值用訓練集中位數補。 - 新增測試集預測圖，含 95% 信賴區間和預測區間。 - **修了兩個 bug**（DIC2 也有，只是那邊資料小沒觸發）： - 特徵尺度差太大時（例如人口數和比例）會被誤判成共線，少算某些 k。 - 瀏覽器內計算太慢：從 38 秒降到 0.9 秒。

檔案（全部未提交）： - `index.html`：網頁 - `app.py`：Flask 後端 - `analysis.py`：產生報告圖表 - `README.md`：CRISP-DM 報告 - `data/Life_Expectancy_Data.csv`：資料集，跟 Kaggle 原檔完全相同 - `figures/`：6 個圖表和結果檔 - 其他照抄 DIC2 的部署設定：`api/index.py`、`vercel.json`、`requirements.txt`、`.gitignore`

你要做的（11/4 截止）： 1. 在 GitHub 建新 repo，例如 `AIoT-HW2-MultipleLR`。 2. Review 後自己 commit / push。commit 訊息草稿：`HW2: multiple linear regression on WHO Life Expectancy (CRISP-DM)` 3. 到 Settings → Pages 開啟（`main` / `/ (root)`），把網址填進 README 第 5 行。 4. 補上 README 最後的「GPT 輔助內容」（附對話 PDF）和「NotebookLM 摘要」（100 字以上）。

資料來源查找： - Kaggle — Life Expectancy (WHO) - aaaastark/Life_Expectancy_Predication（GitHub 副本） - Medium：EDA on WHO Life Expectancy dataset

我

幫我寫 NotebookLM 摘要
老師期望要做的很完整又更多東西他說時間比較常要更完整

Claude（AI）

NotebookLM 摘要這一項，我不能幫你寫好再標成「NotebookLM 摘要」。作業要你**實際用 NotebookLM 研究**，而且報告要分清楚 GPT 輔助和 NotebookLM 的內容。我寫的東西標成 NotebookLM 產出，就是在來源上造假，老師如果要你展示 NotebookLM 筆記本也拿不出來。

不過可以讓你 10 分鐘內做完：素材我已經找好了，你只要貼進去、問問題，產出後我再幫你潤稿、檢查數字。

步驟 1：在 NotebookLM 建新筆記本，加入這些來源（Add source → Website）

來源	重點
NYC Data Science：Modeling Life Expectancy	同一份資料做 MLR，用逐步選擇 + 領域判斷刪共線變數，最後留 10 個特徵
Wenshan Zhang（RPubs）	MLR + 迴歸假設檢查，處理異質變異

來源	重點
Dolgopolyi et al.（arXiv 2510.00542）	比較線性迴歸、決策樹、隨機森林
Intersect（Stanford）	特徵選擇後比較 MLR / RF / DT
IJOE：Immunization and HDI	疫苗與 HDI 因素對壽命的影響
你的 `README.md`	讓 NotebookLM 拿你的結果跟別人比

步驟 2：在聊天框問這個問題

```
請用繁體中文，以 200 字左右摘要這些研究如何用 WHO Life Expectancy 資料集做迴歸分析：
1. 他們用了哪些特徵選擇方法、最後保留哪些特徵
2. 線性迴歸的 R² 或誤差大約多少，和其他模型比較如何
3. 他們怎麼處理缺值與共線性（例如 infant deaths 和 under-five deaths）
4. 和我的 README 結果比較，有哪些相同與不同的發現
```

步驟 3： 把回答貼回來給我。我幫你潤稿成報告格式、核對數字和你的結果一致，再放進 README 的「NotebookLM 摘要」段落。另外截一張 NotebookLM 畫面當證據。

老師說時間比較長要更完整，這部分我可以直接做。你想加哪些？

我

（選項回答）要加哪些內容讓作業更完整？（可複選） → 修改UI 要更美觀,迴歸五大假設檢定 (Recommended),Ridge / Lasso 模型比較 + 交叉驗證,改善模型：轉換 + 離群值,一頁式資訊圖表

Claude（AI）

接著擴充 `analysis.py`：加入五大假設檢定、Ridge/Lasso 交叉驗證、log 轉換與 Cook's distance 離群值，輸出圖表和給網頁用的 `advanced.json`：

分析在背景跑（這台電腦載入 Python 套件很慢，要等幾分鐘）。同時改網頁，先看目前的 HTML 結構：

Python 那邊還在跑。接著改 JS：五大假設檢定（JS 版）、資料理解圖表、資料準備步驟、頂部數字、預測器的預測區間、導覽列跟著捲動標示：

先修正剛寫的腳本：導覽列那段我多寫了一段沒用的重複程式碼，拿掉再執行。

等待期間，用 Python 內建的靜態伺服器模擬 GitHub Pages（只有前端、沒有後端），測試新版網頁：

每一段都正常顯示。接著把 JS 的假設檢定數字跟 statsmodels 比對：切到同一個模型（RFE k=16）：

JS 版的假設檢定數字跟 statsmodels **完全一致**：DW 0.7606、JB 280.03、BP 298.17、偏態 −0.2072、峰態 4.6425、VIF 200.78。

看一下新版面的樣子：

畫面看起來被裁切，檢查是否超出寬度：

上一步的修改其實沒套用到（比對字串失敗），所以重跑時錯誤照舊。改用 Edit 工具直接修：

等待期間做一頁式資訊圖表 `infographic.html`。數字從 `figures/advanced.json` 讀取，相關係數用已核對過的值：

資訊圖表寫好了。裡面有兩句話還要用數據驗證：「CV 從 4.08 跳到 6.04」和「低壽命國家誤差較大」。等分析結果：

趁重跑時，用網頁驗證另一句話「低壽命國家誤差較大」：把訓練集殘差依預測值分組，看變異大小：

接著把報告（README）的「評估」章節補完整，加入五大假設、交叉驗證、正則化比較、模型改善：

Figures regenerated. Checking the web page's evaluation section and the infographic end to end:

進階評估區出現了。截圖檢查相關係數圖、五大假設、進階區：

這些錯誤都是主頁的（找不到後端 API 是預期中的）。直接在資訊圖表頁執行看看哪裡出錯：

內容其實已經載入了，剛才讀文字時 JSON 還沒抓完（或瀏覽器快取到舊的 NaN 版本）。再讀一次確認：

全部數字都正確。修正「12 個欄位」，再完整重跑一次 `analysis.py`，確認 NaN 修正後腳本能從頭跑完，產生的 JSON 也合法：

腳本可以從頭跑完，JSON 也沒有 NaN 了。關掉測試用的靜態伺服器（只關這一個程序）：

五項擴充都做完了，照你的規則沒有 commit / push。網頁上的每一段都用模擬 GitHub Pages 的本機伺服器載入檢查過。五大假設的數字跟 statsmodels 逐項核對，完全一致。`analysis.py` 也從頭重跑過，能正常跑完。

NotebookLM 摘要我沒有代寫。那一段必須是你實際用 NotebookLM 產生的，上一則訊息有來源清單和提問範本，你把結果貼回來，我再幫你潤稿和核對數字。

這次新增的內容：

項目	結果
UI 改版	頂部有 CRISP-DM 六步驟導覽列，捲動時會標出目前段落。頁面依六個階段分區，新增資料理解（缺值、相關係數圖）和資料準備步驟清單
五大假設檢定	五項在大樣本下都被拒絕。報告解釋了各自原因，例如同一國家 16 年的資料彼此相關，所以 DW 只有 0.76
交叉驗證	5 折 CV 曲線跟測試集一致，k=16 不是運氣好選到的
Ridge / Lasso	沒有比 OLS 好。樣本 2,342 筆遠多於 20 個特徵，本來就沒什麼過度擬合
log 轉換 + 離群值	對 10 個偏態特徵取 log，**RMSE 4.07 → 3.62（降 11%）**，是所有嘗試裡最有效的。移除 142 個影響點反而略差，所以不刪
資訊圖表	`infographic.html`，主網頁「部署」區有連結
報告	README 新增 5.1–5.6 節：假設檢定、交叉驗證、正則化、模型改善、踩到的坑、結論

過程中我修正了自己的兩個錯誤： - **錯誤的推論**：我原本說「人口極端值讓交叉驗證 RMSE 跳升」。其實 scikit-learn 對尺度差 10^9 倍的資料會截掉資訊，同一個模型被算成 RMSE 6.80（正確是 4.07），跳升是計算假象。改成先標準化再擬合後就正常了。上次 Python 和 JS 在 Forward k=18 選得不一樣，也是這個原因，現在兩邊一致。這件

事寫進了報告 5.5 節。 - **誇大的說法**：網頁說「全部 20 個特徵反而 overfitting」，但實際只差 0.01 歲，已改成「誤差幾乎一樣」。

檔案（全部未提交）： `index.html` 、 `app.py` 、 `analysis.py` 、 `README.md` 、 `infographic.html` 、 `data/` ，以及 `figures/` 裡的 13 個圖表和結果檔。

你還要做的： 1. 用 NotebookLM 產生摘要，貼回來給我潤稿。 2. 補 README 的「GPT 輔助內容」，並附上對話 PDF。 3. Review 後自己 commit / push，再開啟 GitHub Pages。commit 訊息草稿： `HW2: WHO life expectancy MLR with CRISP-DM report, assumption tests, CV, Ridge/Lasso, log transform`

我

我現在要幹嘛

Claude（AI）

作業 11/4 截止，還剩兩天。建議照下面順序做，每步大概 10–20 分鐘。

1. 先建 repo、推上去（讓 GitHub Pages 先跑起來）

1. 到 GitHub 建一個新 repo，名字 `AIoT-HW2-MultipleLR` ，設 Public，**不要勾**建立 README。
2. 先 review 檔案，沒問題再在專案資料夾依序執行：

```
git add .
```

```
git commit -m "HW2: WHO life expectancy MLR with CRISP-DM report, assumption tests, CV, Ridge/Lasso, log t
ransform"
```

```
git remote add origin https://github.com/Ollyye16/AIoT-HW2-MultipleLR.git
```

```
git push -u origin main
```

1. 到 repo 的 **Settings → Pages**，Source 選 `Deploy from a branch` ，分支選 `main` 、資料夾 `/ (root)` ，按 Save。等 1–2 分鐘就會出現網址，網址是 `https://ollyye16.github.io/AIoT-HW2-MultipleLR/` 。

2. 做 NotebookLM 摘要

1. 打開 notebooklm.google.com，建一個新筆記本。
2. 加入來源：上傳 `README.md` ，再加這 5 個網址：
3. https://nycdatascience.com/blog/student-works/life-expectancy-by-country/
4. https://rstudio-pubs-static.s3.amazonaws.com/671846_62635c28b95045bb984c75c87fb7d415.html
5. https://www.arxiv.org/pdf/2510.00542
6. https://ojs.stanford.edu/ojs/index.php/intersect/article/view/3445
7. https://online-journals.org/index.php/i-joe/article/view/33315
8. 在對話框輸入：

請用繁體中文，以 300 字左右摘要這些資料：影響各國平均壽命的主要因素有哪些？多元線性迴歸在這個資料集上的表現與限制是什麼？我的分析結果（README）和其他研究有哪些一致或不同之處？

9. 把回答**整段複製貼給我**。

3. 準備 GPT 輔助內容的 PDF

在對話頁面按 Ctrl+P，選「另存為 PDF」，存成 `docs/chat.pdf` 放進專案資料夾。可以用這次跟我的對話，也可以用你和 ChatGPT 的對話。

4. 把結果交給我

把 NotebookLM 的回答貼過來，並告訴我 PDF 的檔名。我會： - 潤稿 NotebookLM 摘要，並核對裡面的數字 - 寫好「GPT 輔助內容」這一節 - 把 README 第 5 行換成 Pages 網址

之後你再 commit、push 一次就完成了。

5. 交作業

到 iLearning 作業頁貼上兩個網址： - GitHub repo：`https://github.com/Ollyye16/AIoT-HW2-MultipleLR` - 網頁：`https://ollyye16.github.io/AIoT-HW2-MultipleLR/`

現在先做第 1 步。 推上去後跟我說，我可以幫你打開 Pages 網址，確認網頁和資訊圖表在線上都正常。

我

PS C:\Users\zaxsc> git add .
fatal: not a git repository (or any of the parent directories): .git
PS C:\Users\zaxsc> git commit -m "HW2: WHO life expectancy MLR with CRISP-DM report, assumption tests, CV, Ridge/Lasso, log transform"
fatal: not a git repository (or any of the parent directories): .git
PS C:\Users\zaxsc> git remote add origin https://github.com/Ollyye16/AIoT-HW2-MultipleLR.git
fatal: not a git repository (or any of the parent directories): .git
PS C:\Users\zaxsc> git push -u origin main
fatal: not a git repository (or any of the parent directories): .git
PS C:\Users\zaxsc>

Claude（AI）

錯誤是因為你在 `C:\Users\zaxsc` 執行指令，但專案在子資料夾 `AIoT-HW2-MultipleLR`。我看過了，那個資料夾已經是 git repo，只是還沒有任何 commit。

先切換到專案資料夾：

```
cd C:\Users\zaxsc\AIoT-HW2-MultipleLR
```

然後在 GitHub 上建好空的 `AIoT-HW2-MultipleLR` repo，再依序執行剛剛那四行：

```
git add .
```

```
git commit -m "HW2: WHO life expectancy MLR with CRISP-DM report, assumption tests, CV, Ridge/Lasso, log transform"
```

```
git remote add origin https://github.com/Ollyye16/AIoT-HW2-MultipleLR.git
```

```
git push -u origin main
```

如果 push 時出現 `repository not found`，代表 GitHub 上的 repo 還沒建好，或名字拼錯了。

我

推好了，幫我檢查 Pages 網址

Claude (AI)

Check the repo itself exists and what's pushed.

程式碼已經成功推上去了：GitHub 上的 `main` 和你本機的 commit 一致（`e18ff5c`），repo 也是公開的。

但網頁網址目前是 404，因為 **GitHub Pages 還沒開啟**。我向 GitHub 查詢這個 repo 的 Pages 設定，回傳的是「Not Found」。

開啟方式： 1. 打開 https://github.com/Ollyye16/AIoT-HW2-MultipleLR/settings/pages 2. **Source** 選 `Deploy from a branch` 3. **Branch** 選 `main`，資料夾選 `/ (root)`，按 **Save** 4. 等 1–2 分鐘，頁面上方會出現「Your site is live at …」

開好之後跟我說，我會重新打開網址，檢查主網頁的計算、圖表和資訊圖表在線上是否都正常。

我

開好了，再檢查一次

Claude (AI)

頁面的假設檢定表顯示的是預設選中的模型（Lasso k=19），數字和 RFE k=16 不同是正常的。再檢查資訊圖表：

網頁都正常。順便把 README 裡的網址佔位符換掉：

GitHub Pages 開好了，主網頁和資訊圖表在線上都能正常顯示。

檢查項目	結果
主網頁	資料（2,928 筆）和結果檔都載入成功，沒有錯誤
計算	線上沒有 Python 後端，所以由瀏覽器內的 JavaScript 計算；最佳模型 RFE k=16，RMSE 4.0642、R^2 0.8288，和本機結果一致
進階區塊	假設檢定、OLS / Ridge / Lasso 比較、交叉驗證都有顯示
資訊圖表	大數字、五大假設、log1p 改善表都正常

主控台有一個 404 錯誤，是我第一次在 Pages 還沒開啟時打開網址留下的，不是網站本身的問題。

網頁上假設檢定表的數字（例如 BP 297.13）和報告裡的（298.17）不同，這是正常的。網頁開啟時預設選的是 Lasso k=19，表格顯示的是這個模型；報告寫的是最佳模型 RFE k=16。

我改了一個檔案： `README.md` 第 5 行原本是網址佔位符，已換成 Pages 網址和資訊圖表網址。這個修改還沒 commit。

下一步： 用 NotebookLM 產生摘要後貼給我，並準備好對話 PDF。我把 README 那兩節補完，你再一起 commit、push 一次就可以交作業。

我

但我有一個問題 我認為你做的有點....難抓重點 我看內容我不知道你到底再寫甚麼

Claude（AI）

你說得對。我把太多技術細節塞在一起，卻沒先講結論。整份作業用白話講其實只有這幾件事：

這份作業在做什麼

> 拿一個國家的資料（教育年數、成人死亡率、HIV 比例.....），猜這個國家的人平均能活幾歲。

步驟	白話說法	結果
1. 建模型	多元線性迴歸就是「每個因素乘一個權重，全部加起來」	—
2. 挑因素	20 個因素不一定都要用，試看看用幾個最準	用 16 個最準；**只用 4 個也差不多準**
3. 準不準	拿模型沒看過的 586 筆資料來考它	**平均猜錯約 4 歲**，能解釋 83% 的差異
4. 範圍	不只給一個數字，還給一個範圍	大約 **±8 歲**，實際值有 94% 落在範圍內
5. 健康檢查	迴歸有五個前提，檢查資料符不符合	五個都不符合，預測可以用，但範圍要保守看
6. 改進	把 GDP、人口這種差距極大的數字取 log 壓小	誤差從 **4.07 降到 3.62 歲**
7. 換別的模型	Ridge、Lasso 是「防止模型想太多」的版本	**沒有變好**，因為資料夠多，原本就沒想太多

一句話結論：教育和收入讓人活得久，成人死亡率和 HIV 讓人活得短。這個模型平均誤差 4 歲，把偏態數據取 log 後能再進步 11%。

我建議這樣改，只調整呈現方式，不動計算和頁面結構： - 網頁和 README 最上面都加「三句話看懂」。 - 每個段落開頭先用一句白話寫結論，例如「⇒ 平均猜錯 4 歲」。 - 統計量表格（p 值、VIF、α…）收進「展開看細節」，預設摺疊。

要兩個都改，還是只改網頁或只改 README？

我

兩個都改 而且UI 也改一下 不太好閱讀

Claude（AI）

我先改網頁：加上「三句話看懂」、在每段開頭放一句白話結論、把統計細表收進可展開的區塊，並把字體放大。改完網頁再改 README。

Now the HTML body sections.

Now the JS that fills these in.

Start a local static server to verify.

Everything renders. Two tweaks: plain-language lines should show 2 decimals, and the page should open on the best method (RFE) so the summary and the "current model" agree.

Good. Reload and screenshot.

Hero reads well. Check the evaluation section and that charts inside the collapsed blocks draw when opened.

The two residual charts inside the collapsed block draw too small, because they're created while hidden. I'll resize them when the block opens.

Charts are fixed. Let me look at the evaluation section.

Looks clean. Quick check that the other datasets still work (50 Startups):

Web page done (the 501s are just the static test server lacking the Flask API — expected). Now README.

Now rewriting the README in the same style: summary first, a plain one-line conclusion for each section, and the technical details in collapsible blocks (GitHub renders `<details>`).

網頁和 README 都改好了。這次只改呈現方式，計算方法和頁面結構都沒動。網頁我在本機跑過 Life Expectancy 和 50 Startups 兩個資料集，都正常；README 還沒看過 GitHub 上的實際顯示效果。

網頁 `index.html` - 最上面先講結論：「三句話看懂」說明在預測什麼、平均猜錯 4.06 歲、哪些因素讓壽命變長或變短。這三句會依資料集自動產生，換成 50 Startups 也一樣有。 - **每一段開頭加一句白話結論**（黃色框），例如：

> 用 16 個因素時誤差最低。但其實只用 4 個就有最佳模型 95% 以上的解釋力。 - **五大假設改成五張小卡**：每張用白話寫出前提，標示「✓ 符合」或「⚠ 不符合」。檢定統計量、p 值和殘差圖收進「▸ 展開看細節」。 - **其他技術表格也預設收起來**：係數表、Ridge/Lasso 比較、交叉驗證圖、猜錯最多的 10 筆，點開才顯示。 - **比較好讀**：字體從 14px 放大到 15px，灰色文字調亮；指標名稱改成白話，例

如「平均猜錯（RMSE）」「能解釋的差異（R^2）」。 - **開頁直接顯示最佳模型**：預設選 RFE k=16。之前預設的是 Lasso k=19，跟摘要寫的最佳模型不一致。

報告 `README.md` - 最前面新增「三句話看懂」和「一張表看完」，用白話說明整份作業在做什麼。 - 每個 CRISP-DM 段落開頭先放一句結論。 - 檢定數字、交叉驗證、Cook's distance 等細節收進可展開區塊。GitHub 會把它顯示成可點開的「▸」。

改到的檔案（未 commit）： `index.html`、`README.md`

你可以先用 VS Code 或瀏覽器打開本機的 `index.html` 看看。我在本機測試時，畫面上的數字沒有問題。確認後照之前的方式 commit / push，GitHub Pages 會在 1–2 分鐘內更新：

```
git add index.html README.md
```

```
git commit -m "Make page and README easier to read: plain-language summaries, collapsible details"
```

```
git push
```

push 之後跟我說，我可以再檢查一次 GitHub Pages 上的網頁。

我

推好了，再檢查一次

Claude（AI）

Also check the README renders the collapsible blocks on GitHub.

線上版本已經更新，GitHub 上的 commit 和你本機一致（`4c11497`），沒有漏推的檔案。

網頁（https://ollyye16.github.io/AIoT-HW2-MultipleLR/）-「三句話看懂」、每段的黃色結論框、五大假設小卡都有顯示，數字正確。 - 開頁預設選的是最佳模型 RFE k=16，和最上面的摘要一致。 - 技術細節都收在「展開看細節」裡。

GitHub 上的 README -「三句話看懂」和「一張表看完」排在最前面。 - 7 個可展開區塊都正常顯示，6 張圖片都有載入。

剩下要做的： 1. **NotebookLM 摘要**：照之前的步驟在 NotebookLM 加入來源並提問，再把回答貼給我。 2. **GPT 輔助內容**：在對話頁面按 Ctrl+P 另存成 PDF，放到 `docs/chat.pdf`。 3. 把這兩樣給我，我會補進 README，你再 commit、push 一次。 4. 到 iLearning 繳交 repo 網址和網頁網址，截止日是 11/4。

我

但我認為UI還是沒有很美觀 配色有點不太舒服

（選項回答）想換成哪種配色？（只換顏色，版面不動）→ 淺色，暖米綠

Claude (AI)

I'll do a straight color swap (no layout change) on both `index.html` and `infographic.html`, mapping each old color to the warm cream/green palette.

All old colors replaced. Preview:

Looks calm. Check the charts and evaluation area:

Points outside the prediction range (red) are hard to tell apart from the brown ones. I'll lighten the in-range points.

Both look good. Cleaning up.

主網頁和資訊圖表都換成「淺色．暖米綠」配色了，版面和內容沒有動。我在本機打開兩個頁面看過，畫面正常。

用途	原本	現在
背景	深藍黑	米白 `#faf8f3`
卡片	深藍	白
主色（按鈕、正相關、預測線）	螢光青綠	墨綠 `#15803d`
重點（黃框結論、最佳 k）	亮黃	棕橘 `#b45309`
負向（負相關、超出預測範圍的點）	螢光粉紅	磚紅 `#b91c1c`
圖表線條	螢光五色	墨綠、紫褐、灰藍、赭紅、橄欖綠

另外調了一個地方：預測圖上，落在預測範圍內的點改成淡棕色，這樣範圍外的紅點比較好認。

改到的檔案（未 commit）： `index.html`、`infographic.html`

確認沒問題的話：

```
git add index.html infographic.html
```

```
git commit -m "Switch to light warm palette for readability"
```

```
git push
```

如果整體還是不喜歡，可以改用另外兩套：淺色．溫和藍，或深色．低飽和。

我

推好了，再檢查一次

Claude (AI)

The new version is live (the earlier "0" was just a grep quirk). Visual check:

線上網頁已經換成新的淺色配色，數字和功能都正常。

- GitHub 上的 `main` 和你本機一致（`556375d`），沒有漏推的檔案。
- 主網頁是米白底、墨綠和棕橘，開頁直接顯示最佳模型 RFE k=16，數字正確，主控台也沒有錯誤。
- 資訊圖表也是新配色。

如果你那邊看到的還是舊的深色版，按 **Ctrl+F5** 強制重新整理。GitHub Pages 最多會快取 10 分鐘。

剩下要做的： 1. **NotebookLM 摘要**：在 NotebookLM 加入那 5 個網址和 `README.md` 當來源並提問，把回答貼給我。 2. **GPT 輔助內容**：在對話頁按 Ctrl+P 另存成 PDF，放到 `docs/chat.pdf`。 3. 把這兩樣給我，我補進 README，你再 commit、push 一次。 4. 11/4 前到 iLearning 繳交 repo 網址和網頁網址。

我

NotebookLM 要怎麼用 哪5個阿?

Claude（AI）

這 5 個是網路上其他人用同一份 WHO 壽命資料做的分析和論文。NotebookLM 會讀它們，再和你的 README 對照、寫出摘要。我剛確認過網址：4 個正常，第 1 個（NYC Data Science）有防爬蟲，NotebookLM 可能匯入失敗，失敗就跳過，剩 4 個也夠用。

步驟

1. 開新筆記本 打開 https://notebooklm.google.com ，用 Google 帳號登入，按「**+ 建立新的筆記本**」。

2. 加入你的報告 在「新增來源」視窗選「**上傳檔案**」，選 `C:\Users\zaxsc\AIoT-HW2-MultipleLR\README.md`。如果不接受 `.md` 檔，改用 GitHub 上的網址：`https://github.com/Ollyye16/AIoT-HW2-MultipleLR`

3. 加入 5 個網址 按左側「**+ 新增**」，選「**網站**」，一次貼一個，按「插入」，重複 5 次：

```
https://nycdatascience.com/blog/student-works/life-expectancy-by-country/
```

```
https://rstudio-pubs-static.s3.amazonaws.com/671846_62635c28b95045bb984c75c87fb7d415.html
```

```
https://www.arxiv.org/pdf/2510.00542
```

```
https://ojs.stanford.edu/ojs/index.php/intersect/article/view/3445
```

```
https://online-journals.org/index.php/i-joe/article/view/33315
```

#	來源	內容
1	NYC Data Science 學生作品	同一份資料的迴歸分析
2	RPubs（R 語言報告）	同資料的多元迴歸與特徵選擇

#	來源	內容
3	arXiv 論文	壽命預測的機器學習研究
4	Stanford Intersect 期刊	影響壽命的社經因素
5	iJOE 期刊	壽命資料的迴歸與模型比較

4. 確認來源都打勾 左側清單每個來源前面的勾勾都要勾上。

5. 在中間對話框貼上這段，按送出：

> 請用繁體中文，以 300 字左右摘要這些資料：影響各國平均壽命的主要因素有哪些？多元線性迴歸在這個資料集上的表現與限制是什麼？我的分析結果（README）和其他研究有哪些一致或不同之處？

6. 把回答整段複製貼給我 我會幫你潤稿，核對裡面的數字有沒有和你的結果衝突，再放進 README 的「NotebookLM 摘要」。

建議截圖： 回答出來後，截一張包含左側來源清單和回答的畫面，存成 `docs/notebooklm.png`。老師要看你真的有用 NotebookLM，截圖是最好的證明，我也會一起放進 README。

我

影響平均壽命的主要因素
各國平均壽命主要受社會經濟與醫療健康兩大面向影響：人類發展指數（HDI/資源收入結構）、受教育年數及 GDP 與壽命呈強正相關；成年人死亡率、HIV/AIDS 感染率與青少年/兒童消瘦率呈顯著負相關；身體質量指數（BMI）及疫苗接種率（如白喉、小兒麻痺）則具正面促進效果。
多元線性迴歸的表現與限制
在測試集上，多元線性迴歸的判定係數 R^2 約為 0.73（MAE 3.74、RMSE 4.82），具備運算極快與參數易解釋的優點。然而其主要限制在於無法有效捕捉特徵間的非線性關係與交互作用，且容易受到離群值干擾。此外，真實資料常難以完全符合殘差常態性、同變異性與無多重共線性等統計前提。
分析結果（README）與其他研究的異同

* 一致處：各研究均證實成人死亡率、HIV/AIDS、GDP 及疫苗率為關鍵預測因子，且非線性模型（如隨機森林 $R^2 \approx 0.94$）的預測準確度均顯著超越線性迴歸。
* 不同處：README 著重於完整的 CRISP-DM 實驗流程、迴歸前提假設檢定與 Ridge/Lasso 正規化比較；而部分研究在預處理時直接剔除缺值率高於 5% 的欄位（如 GDP 與教育年數），導致特徵重要性排序與涵蓋完整 HDI 變數的研究產生差異。

💡 想進一步了解如何透過 Ridge/Lasso 正規化改善迴歸共線性，或是為這個實驗整理出一份完整的簡報嗎？

Claude（AI）

NotebookLM 的回答有兩處需要核對，我先把內容放進 README，順便存下你的截圖。

NotebookLM 摘要已經放進 README 的「NotebookLM 摘要」那一節，你的截圖也存成 `docs/notebooklm.png` 一起放進去了。

我怎麼處理這段內容： - **NotebookLM 的回答保留原文**，只把 `\(R^2\)` 這類公式符號改成一般文字 R²。這段是作業要求的 NotebookLM 產出，改寫內容就不算它寫的了。 - **另外加了「我的核對與心得」**，處理兩個容易誤會的地方： 1. 回答裡的「R² 0.73、RMSE 4.82」是**其他研究**的結果，不是你的。你的是 R² 0.83、RMSE 4.06 歲，取 log 後是 R² 0.86、RMSE 3.62。不註明的話，老師可能以為你的數字前後矛盾。 2. 回答說「非線性模型更準」是你的結果也一致的地方，但你沒有訓練隨機森林。我改成說明方向一致的實際依據：RESET 檢定顯示資料還有非線性，取 log 這個非線性轉換也讓誤差降了 11%。 - **註明來源**：NYC Data Science 那篇在你的截圖裡沒有勾選、旁邊有驚嘆號，表示沒匯入成功。我在 README 寫明它被網站阻擋，實際用的是其他 5 個來源。

改到的檔案（未 commit）： `README.md`、`docs/notebooklm.png`（新增）

```
git add README.md docs/notebooklm.png
```

```
git commit -m "Add NotebookLM summary with fact-check notes"
```

```
git push
```

最後一項：GPT 輔助內容 1. 把對話匯出成 PDF。作業提到用 pdfCrowd，也可以在對話頁按 Ctrl+P →「另存為 PDF」。 2. 存到 `C:\Users\zaxsc\AIoT-HW2-MultipleLR\docs\chat.pdf`。 3. 跟我說一聲，我會寫好 README 的「GPT 輔助內容」，說明哪些部分有用 AI 協助，並附上 PDF 連結。

做完這項就可以到 iLearning 交作業了。

我

GPT 輔助內容 怎麼用

Claude（AI）

「GPT 輔助內容」是要你**誠實交代哪些部分用了 AI**，並附上對話紀錄 PDF 當證明。你用的是 Claude，不是 ChatGPT，照實寫 Claude 就可以，老師要看的是 AI 怎麼被使用。

README 這一節會有兩樣東西： 1. **說明 AI 幫了什麼**，我會照實際狀況寫： - 程式：網頁、Flask 後端、`analysis.py` - 統計解釋：五大假設、交叉驗證的意義 - 除錯：scikit-learn 尺度問題 - 你自己做的部分：選資料集、決定要做哪些擴充、指出 UI 不好讀並要求修改、跑 NotebookLM、review 並 commit 2. **附上對話紀錄 PDF 的連結**。

PDF 有兩種做法：

做法	怎麼做	缺點
A. 我幫你產生（推薦）	我讀取本機這段對話的紀錄檔，排成好讀的「你問 / AI 答」網頁，再用 Edge 轉成 `docs/chat.pdf`	對話很長，PDF 可能幾十頁
B. 自己匯出	這個 App 的 Code 分頁沒有 Ctrl+P 列印整段對話的功能，要一頁一頁截圖或複製貼上	很麻煩

做法 A 只會放進對話文字，不會放程式執行輸出的大量細節，也不會放你的 email 或帳號。你回「好」，我就開始產生 PDF 並寫好 README 這一節。完成後你看過 PDF 沒問題再 commit。

我

好，幫我產生 PDF

Claude（AI）

PDF 的對話內容已經整理出 55 段，現在補上你在選項框裡做的選擇，並檢查個人資料。

Update the script to include those answers and redact personal data.